

Dependency-order differential compression of the gapped suffix array

1 Setup and preliminaries

1.1 Gapped k -mers and DisLex

Fix a binary shape S of *span* s and *weight* w (number of care positions). On a DNA text $T[0..n)$, every start position p yields a gapped k -mer name $c = \text{name}_S(T, p)$. For the running example we take

$$T = (\text{GCCTTTAAAG})^3, \quad |T| = n = 30, \quad S = \# . \#, \quad s = 3, \quad w = 2.$$

The **DisLex** transform (Horton) lays these names out grouped by residue class $p \bmod s$, producing an integer string (the *lertext*) of length $m = n + 1 = 31$. Consecutive symbols inside a residue class are exactly the successive windows $p, p + s, p + 2s, \dots$ — i.e. the gapped suffix starting at p . The suffix array of the lexttext is the **gapped suffix array** (GSA); an LCP array is computed in symbol space (Kasai). Writing $\text{sa}[r]$ for the lexttext rank- r suffix and $\text{orig}(r)$ for its original text start, the first symbol of that suffix is the gapped k -mer at $\text{orig}(r)$.

1.2 LCP intervals I_c

Because the GSA is ordered by gapped suffixes, all ranks whose first symbol equals a fixed name c form a contiguous interval

$$I_c = [\ell_c, r_c) \subseteq \{0, \dots, m - 1\}.$$

The multiset $\{\text{orig}(r) : r \in I_c\}$ is precisely the sorted list of text positions of c . On the example, the intervals with $|I_c| > 1$ are:

c	SA range I_c	$ I_c $	text positions $\text{orig}(\cdot)$ (rank order)
AA	[2, 5)	3	26, 16, 6
AG	[5, 10)	5	27, 18, 8, 17, 7
CT	[10, 16)	6	22, 21, 12, 2, 11, 1
GC	[17, 22)	5	19, 9, 20, 10, 0
TA	[22, 28)	6	25, 24, 15, 5, 14, 4
TT	[28, 31)	3	23, 13, 3

(Singletons $\$ \$$, $\text{A\$}$, $\text{G\$}$ are never differentially compressed.)

1.3 Differential links

Let $x = \text{sa}[r]$ be a lexttext position. Shifting left by a symbols in the lexttext corresponds to walking a windows backward along the same residue class. Whenever the alignment condition

$$\text{lex2orig}[x - a] = \text{orig}(r) - a \cdot s$$

holds, the suffix at $x - a$ is a valid **left context** of depth a . Prepending a symbols preserves relative order among suffixes and shifts original positions by $+a \cdot s$.

Consequently, if a contiguous SA run $J = [\sigma, \tau)$ has LCP depth at least $a + 1$ and the symbol at depth a equals c , then mapping each kept source rank $u \in J$ by

$$\text{orig}(u) \mapsto \text{orig}(u) + a \cdot s$$

recovers, bijectively and in order, a subset of I_c . We call (J, a) a **differential source** of coverage $\text{cov} = |J'|$ for those covered ranks of I_c .

1.4 What is stored: C , H_{offset} , H_{rest}

Compression drops covered ranks from the stored position array. The kept original positions, still in GSA rank order, form the compressed array C . For each name c the hash table stores at most two entries:

- $H_{\text{offset}} = (\text{pos}, a, \text{num})$: recover num positions as $C[\text{pos} + t] + a \cdot s$ for $t = 0, \dots, \text{num} - 1$;
- $H_{\text{rest}} = (\text{pos}, \text{num})$: num positions stored directly in C .

Ranks used as sources of an accepted link are **pinned** (their pin-count increases) so that later decisions cannot remove them while dependents still need them. The objective of source selection is to minimise $|C|$.

2 The preference relation

For each interval I_c with $|I_c| > 1$, the implementation enumerates all viable sources for $a = 1, \dots, a_{\text{max}}$ (default $a_{\text{max}} = 8$):

1. For every rank $r \in I_c$, compute the predecessor lertext coordinate $y = x - a$ if the alignment condition above holds; collect pairs $(\text{rank}(y), r)$.
2. Sort by source rank and group into maximal contiguous runs where consecutive source ranks differ by 1 and $\text{LCP} \geq a + 1$.
3. Retain a run only if it is **disjoint** from I_c and has coverage at least 2.

Among all such candidates, the **preferred** candidate for I_c maximises coverage, breaking ties by preferring larger a (deeper LCP). Availability (pins / already-removed ranks) is *ignored* when building preferences: preference is a global, static notion of “who would like whom as differential source.”

Directionality is one-sided: sources always lie to the left in the lertext (earlier windows on the same residue class). There is no “forward” differential link.

3 The DepOrder algorithm

DepOrder has three conceptual stages.

3.1 Preference DAG

For every interval with a preferred candidate of coverage ≥ 2 , draw a directed edge

$$\text{source name} \longrightarrow \text{target name}$$

meaning “the target prefers a differential source inside the source’s interval.” Self-loops are discarded. The resulting digraph may contain cycles.

On the example the preferred candidates are:

target	prefers	a	cov	source SA range	covered ranks in target
AA	GC	2	3	[19, 22)	{2, 3, 4}
AG	CT	2	5	[11, 16)	{5, 6, 7, 8, 9}
CT	GC	4	2	[20, 22)	{10, 12}
GC	CT	3	2	[14, 16)	{19, 20}
TA	CT	1	6	[10, 16)	{22, ..., 27}
TT	GC	1	3	[19, 22)	{28, 29, 30}

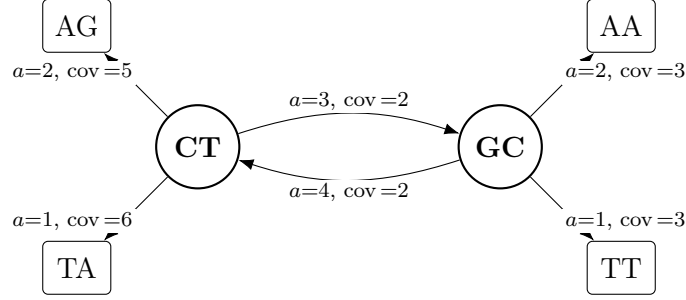


Figure 1: Preference DAG for $T = (\text{GCCTTTAAAG})^3$, $S = \#.\#$. An edge $w \rightarrow c$ means that c 's preferred differential source lies inside I_w . Hubs CT and GC form a mutual-preference cycle; leaves AA, TT, AG, TA are high-coverage dependents.

Hubs. Both CT and GC are preferred sources for three names each. There is a mutual preference cycle $\text{CT} \leftrightarrow \text{GC}$. The high-value dependents are the leaves AA, TT (via GC) and AG, TA (via CT): together they offer coverage $3 + 3 + 5 + 6 = 17$, while the hub–hub edges themselves offer only coverage 2 each.

3.2 Phase I — reverse topological acceptance (sinks first)

A Kahn topological order is computed on the preference DAG; remaining nodes in any strongly connected component (here the CT–GC cycle and its waiting dependents) are appended by decreasing interval size. **Acceptance proceeds in reverse topological order:** dependents / sinks before sources.

For each name in that order, the algorithm recomputes the best candidate *under current availability* (source run must be fully kept; covered ranks must be free of pins and not already removed) and, if coverage ≥ 2 , accepts it: pin the source ranks, mark covered ranks removed.

Why sinks first helps hubs. Accepting a leaf that points into a hub pins the hub's ranks before the hub itself is considered. A hub that would like to compress *itself* away (using the other hub) then finds its own ranks already pinned and is forced to **skip**, remaining available as a source for all dependents. Size-first greedy does the opposite: it lets a large hub claim a weak slice of another hub early, spoiling high-coverage leaf links.

3.3 Phase II — unpin / retarget with transitive add

After Phase I, a fixed-point pass repeatedly tries, for intervals in decreasing size order, candidates of *strictly larger* coverage than the current acceptance — temporarily ignoring pin constraints on covered ranks. Accepting such a candidate may require **retargeting** dependents that currently pin those ranks: if a dependent D 's source run is a contiguous sub-run of the new candidate's covered ranks, D is rewritten to point at the corresponding sub-run of the new source with

$$a'_D = a_D + a_{\text{new}}.$$

The attempt is transactional (snapshot / rollback) and kept only if the global number of kept ranks decreases, or coverage for that name improves. Phase II is where former sources can themselves be compressed once their dependents have been lifted onto a deeper source.

4 Worked example: GCCTTTAAAG $\times$ 3 with #.#

Parameters: $n = 30$, $s = 3$, $m = 31$, $a_{\max} = 8$. Distinct names: 9.

4.1 Phase I on the example

Because of the CT–GC cycle, Kahn’s order first drains the indegree-0 singletons, then appends the cyclic component by size. Reverse-topo acceptance among compressible names is

TT, AA, GC, AG, TA, CT.

step	interval	decision	detail
I.1	TT	ACCEPT	$\leftarrow$ GC[19, 22), $a = 1$, cov=3. Pins GC ranks 19–21.
I.2	AA	ACCEPT	$\leftarrow$ GC[19, 22), $a = 2$, cov=3. Same hub pins reinforced.
I.3	GC	SKIP	Preferred source CT[14, 16) would cover GC ranks {19, 20}, but those ranks are already pinned by TT/AA. Availability blocks self-compression of the hub.
I.4	AG	ACCEPT	$\leftarrow$ CT[11, 16), $a = 2$, cov=5. Pins CT ranks 11–15.
I.5	TA	ACCEPT	$\leftarrow$ CT[10, 16), $a = 1$, cov=6. Pins full CT hub.
I.6	CT	SKIP	Preferred GC[20, 22) is pinned; covered CT ranks are themselves pinned as sources. Hub preserved.

After Phase I: **kept** =14, four accepted offsets (AA, AG, TA, TT). Phase II finds no improving retarget on this instance, so the final size remains $|C| = 14$.

Explicitly, C stores the three singletons plus the entire hubs CT and GC:

$$C = [30, 28, 22, 21, 12, 2, 11, 1, 29, 19, 9, 20, 10, 0].$$

Hash table (offset / rest):

c	H_{offset}	H_{rest}
\$\$	—	(0, 1)
A\$	—	(1, 1)
AA	(11, 2, 3)	—
AG	(3, 2, 5)	—
CT	—	(2, 6)
G\$	—	(8, 1)
GC	—	(9, 5)
TA	(2, 1, 6)	—
TT	(11, 1, 3)	—

All 17 non-hub, non-singleton positions are recovered from the two hubs: $3 + 5 + 6 + 3 = 17$, and $31 - 17 = 14$.

4.2 Contrast: size-order greedy ($|C| = 22$)

Greedy processes intervals by decreasing $|I_c|$, with availability, and pins sources permanently:

1. **CT** (size 6) accepts GC[20, 22), $a = 4$, cov only **2** — a weak use of two GC hub ranks to drop two of six CT positions.
2. **TA** (size 6) is then limited to the same spoiled GC slice: cov only **2**.
3. **AG** (size 5) **skips** — its preferred CT source is no longer fully available.
4. **GC** falls back to TT[29, 31), $a = 2$, cov **2**.
5. **AA** still accepts GC[19, 22), cov 3.
6. **TT skips**.

Net removal: $2 + 2 + 2 + 3 = 9$ ranks, hence $|C| = 22$. The structural mistake is clear: by letting the hub CT claim a low-coverage fragment of GC first, greedy destroys the high-coverage leaf edges (AG cov 5, TA cov 6, TT cov 3) that DepOrder’s sinks-first order realises in full.

algorithm	$ C $	fraction of $m = 31$
Greedy	22	71.0%
DepOrder	14	45.2%

(On this instance Greedy-DFS reaches 15 — better than greedy, still one above DepOrder.)

5 Complexity and when DepOrder helps

Phase I is a single pass over a topological (or size-refined) order, each step re-enumerating candidates for one interval — inexpensive relative to GSA construction. Phase II may repeat enumeration, snapshotting, and retarget attempts over $O(N)$ rounds ($N = \#$ intervals) and is the asymptotic bottleneck on large instances; on the present toy example it is a no-op.

DepOrder helps precisely when the preference graph has **high fan-out hubs** whose own preferred self-compression edges are low-coverage, while many dependents offer large coverage into those hubs. Sinks-first pinning protects the hubs; Phase II’s transitive add is reserved for the harder case where a hub should later be compressed *through* a deeper source after dependents have been retargeted. The GCCTTTAAAG $\times 3 / \# . \#$ instance is the minimal didactic witness of the hub-protection phenomenon: CT and GC must stay in C , and every other multi-occurrence name collapses onto them, yielding $|C| = 14$ against greedy’s 22.